VR-BASED ADAPTIVE GERMAN SUPPORT SYSTEM

Group B — AI Support
Final Submission Report

Scenario-based language support for international HSRW students at A2 German level

Hochschule Rhein-Waal (HSRW)
August 2026

Name	ID
Syed Abuzar Hashmi	34393
Murad Zulfugarov	36296
Muhammad Sufyan	34441
Md. Shaiful Azam	36068
Ubaid khan	34482
Víctor Juárez Gil	35433

Repository: github.com/UbaidKhanGit/vr-adaptive-german-support-system

Table of Contents

Abstract

This report presents the AI Support contribution to a VR-based adaptive German support system for international students at Hochschule Rhein-Waal (HSRW) learning German at approximately A2 level. The project uses realistic everyday scenarios to prepare learners for communication situations that can be difficult in Germany. The first scenario is a doctor's visit, where learners practise describing symptoms, understanding questions, and requesting relevant documents.

The AI prototype is implemented as a Python FastAPI server. It accepts raw 16-bit PCM audio, uses faster-whisper for German speech-to-text, applies a rule-based DoctorBrain dialogue engine to the transcribed input, and uses Argos Translate for German-to-English translation of the learner transcript. The server returns a structured JSON response containing the German transcript, English transcript, and doctor dialogue responses. The repository also contains a standalone terminal prototype and documentation for the API and architecture.

The report distinguishes the current implementation from the intended future adaptive system. While the Unity VR client is fully implemented and capable of sending audio / receiving responses, the AI endpoint remains single-shot rather than streaming, and sensor information is not yet connected to the AI endpoint as a complete adaptive loop. These limitations are important because the current deliverable is a working AI-language prototype and architectural foundation, rather than a fully integrated production system.

Keywords

AI support; German A2; VR language learning; speech recognition; dialogue systems; translation; FastAPI; sensors; adaptive learning.

Executive Summary

- **Problem:** international students may need practical German communication skills for high-stakes everyday situations.
- **Approach:** combine immersive scenarios with spoken interaction and AI-supported language processing.
- **Current implementation:** FastAPI AI server, faster-whisper STT, DoctorBrain dialogue engine, Argos Translate, API documentation, terminal prototype, and implemented Unity VR client.
- **Planned integration:** sensor-informed adaptation.
- **Key limitation:** the current system does not yet implement the complete end-to-end adaptive VR loop.

List of Figures

List of Tables

1. Project Context and Objectives

1.1 Project context

The overall project proposes a student-centred language-support platform for international HSRW students. The system focuses on practical communication rather than abstract grammar exercises. The project description identifies several everyday contexts, including medical communication, travel, Foreigners' Office interactions and accommodation-related situations. The doctor scenario is the main concrete scenario used by the current AI prototype.

1.2 Target users

Aspect	Description
Primary users	International HSRW students learning German at approximately A2 level.
Learning need	Practical spoken communication for everyday situations in Germany.
First scenario	Doctor's office: symptoms, questions, appointment-related communication and sick leave.
Support mode	Audio, text/visual assistance, dialogue practice and future adaptive support.

1.3 Objectives of Group B

- Provide an AI backend that can receive learner speech and return structured language responses.
- Convert spoken German into text so that it can be processed by the dialogue system.
- Maintain scenario-specific conversation logic for the doctor use case.
- Provide German-English language assistance for learners who need additional support.
- Create an architecture that can later connect the AI service with VR and sensor components.
- Document the current implementation, interfaces, limitations and future integration

requirements.

1.4 Scope

This report concentrates on the AI Support contribution. It does not claim ownership of the full VR, interface, sensor, human-factors or ethics work. Cross-group dependencies are described only where they affect the AI architecture.

2. AI Support Requirements

The AI component was derived from the overall project goal of supporting A2 learners in realistic communication. The requirements below describe the intended behaviour while clearly separating requirements from the current implementation.

2.1 Functional requirements

ID	Requirement	Description
FR-01	Speech input	Accept learner speech as audio and process it as German speech.
FR-02	Transcription	Return a German transcript suitable for dialogue processing.
FR-03	Dialogue	Select a context-appropriate doctor response from the current dialogue model.
FR-04	Translation	Provide English translation of the learner transcript for subtitle/support purposes.
FR-05	Structured response	Return a predictable JSON

ID	Requirement	Description
		payload for the client.
FR-06	Error handling	Return a clear response when audio is missing or processing fails.
FR-07	Extensibility	Allow future integration with VR and sensor components.

2.2 Learning requirements

Requirement	Design implication
A2 appropriateness	Keep scenario language accessible and practical; avoid unnecessary linguistic complexity.
Immediate support	Use hints, translation or suggested phrasing when learners cannot express an intended message.
Context	Doctor dialogue should respond to symptoms and conversational intent rather than isolated vocabulary only.
Practice	Learners should be able to repeat a scenario and improve through repeated attempts.
Confidence	The system should support learners without treating mistakes as failure.

2.3 Non-functional requirements

- Predictable API behaviour for integration with a client.
- Clear separation between the VR client and backend language processing.
- Graceful degradation when translation fails.
- Explicit documentation of current limitations and security/privacy considerations.
- A design that can later support session-specific state rather than a single global dialogue state.

3. System Architecture

The repository documentation describes two principal technical parts: a Unity VR client and a Python AI server. The client records spoken audio in the virtual doctor's office and sends it over HTTP to the AI server. The Unity VR client is implemented and capable of sending audio / receiving responses. The AI server is the implemented backend component.

Figure 1. Current AI-support architecture and planned integration points.

3.1 Component responsibilities

Component	Current role	Status
VR client	Capture microphone audio; display/speak AI response in VR. Implemented and capable of sending audio / receiving responses.	Implemented
AI server	FastAPI backend; speech recognition, dialogue and translation.	Implemented prototype
DoctorBrain	Rule-based doctor dialogue and symptom flows.	Implemented
Argos Translate	German-to-English	Implemented

Component	Current role	Status
	transcript translation; DE↔EN packages loaded.	
Sensors	Sensor-related project component for future contextual adaptation.	Separate component; integration incomplete
Documentation	Architecture, API and repository guidance.	Implemented

3.2 Design principle

The architecture intentionally separates presentation and language processing. This means the AI service can be tested independently from Unity/VR, while the client can treat the AI server as an HTTP service. The separation also creates a clear interface for later sensor-aware adaptation.

4. AI Server Implementation

The current AI backend is implemented in Python using FastAPI. FastAPI provides the HTTP application layer and supports explicit route definitions and request handling. The active application exposes two implemented endpoints: POST /translate-audio and POST /reset.

4.1 Startup process

1. The server starts a FastAPI application with a lifespan function.
2. Argos Translate German→English and English→German language packages are ensured at startup.
3. A faster-whisper 'small' model is loaded once and stored in application state.
4. The server is then ready to accept audio requests.

4.2 Request processing

Figure 2. Implemented single-request AI data flow: speech recognition, dialogue response and

learner-transcript translation.

4.3 Audio format

The API documentation specifies raw binary audio as 16-bit signed PCM, mono, 16 kHz sample rate. The server converts the incoming bytes to a NumPy float32 array and normalises the samples before passing the audio to faster-whisper.

4.4 Error behaviour

Condition	Current response
Empty request body	400 Bad Request with 'No audio data received'.
Processing exception	500 Internal Server Error with exception detail.
No speech recognised	Fixed German/English repeat request is returned.
Translation failure	Transcript remains available and translated field becomes '[translation unavailable]'.

5. Speech Recognition

5.1 Role of speech recognition

Spoken interaction is central to the doctor scenario because the learner is practising communication that will eventually happen orally. Speech recognition converts the learner's audio into text that the dialogue engine can analyse.

5.2 Current implementation

The server initialises faster-whisper using the 'small' model on CPU with int8 computation. During a request, the audio is passed directly to the transcription model and the language is

specified as German. The resulting segments are joined into the german_transcript value.

audio bytes → NumPy int16 samples → float32 normalisation → faster-whisper → German transcript

5.3 Example

Stage	Example
Learner speech	Ich habe Kopfschmerzen.
Speech recognition	Ich habe Kopfschmerzen.
Intent/symptom detection	kopfschmerzen
Dialogue response	Nehmen Sie schon Schmerzmittel dagegen?

5.4 Limitations

Speech recognition quality for non-native accents, background noise, microphone differences and short utterances has not been established through a large target-user study. This should therefore be treated as a prototype capability rather than a validated accuracy claim.

6. DoctorBrain Dialogue Engine

DoctorBrain is the current scenario-specific dialogue engine implemented inside ai-server/main.py. It is important to describe it accurately: it is a rule-based dialogue engine, not a generative large language model.

6.1 Normalisation and intent matching

The implementation lowercases and normalises learner text, removes punctuation, maps supported digit words, and then compares the resulting word set with predefined intent and symptom vocabularies.

6.2 Multi-step symptom flows

For selected symptoms, DoctorBrain enters a multi-step flow. The current code contains detailed flows for fever, headache and cough, together with simpler one-off symptom prompts for dizziness, tiredness, back pain, nausea and general pain.

Symptom/intent	Example next question or behaviour
Fever	Asks when the fever started and about measured temperature.
Headache	Asks about pain medication, pain location and hydration/sleep.
Cough	Asks whether the cough is dry or productive, timing and smoking.
Dizziness	Asks about water intake and sleep.
Tiredness	Asks about hours of sleep.
Back pain	Asks about prolonged sitting.
Nausea	Asks about vomiting.
Sick note	Can recognise krankschreibung, attest and related terms.

6.3 Dialogue state

The implementation maintains a DoctorBrain object with the current flow, step index, completed flows and generic response index. The current server creates one global instance. The architecture documentation therefore identifies session scoping as a limitation: concurrent users would share the same conversation state.

6.4 Fallback behaviour

If no specific symptom or intent is recognised, DoctorBrain returns one of several generic doctor responses. If the speech transcript is empty, the server instead asks the learner to repeat the statement.

7. Translation and Language Assistance

Argos Translate is used as the translation component. The server ensures German-to-English and English-to-German packages are available at startup. In the current FastAPI request path, the German learner transcript is translated into English for subtitle/support purposes.

7.1 Current request path

german_transcript → Argos Translate (de → en) → user_transcript_en

7.2 Why translation is useful

- An A2 learner may know the intended meaning but not the German expression.
- An English subtitle can help the learner understand what the system interpreted.
- Translation can support assisted practice without replacing the learner's attempt.
- The standalone terminal prototype also demonstrates English-to-German assistance for pronunciation practice.

7.3 Important implementation detail

The API documentation clarifies that the doctor's bilingual response is stored directly in the DoctorBrain data. Argos Translate is used in the request path to translate the learner's German transcript into English. Therefore, the report does not claim that the doctor's reply is machine-translated by Argos in the current endpoint.

7.4 Graceful degradation

If translation fails, the endpoint does not discard the successful speech recognition and dialogue result. Instead, the translated transcript field is returned as [translation unavailable]. This is a useful resilience pattern because a secondary translation service failure does not necessarily stop the primary conversation.

7.5 Limitations

Argos Translate runs offline, so no network or API key is needed and no learner audio leaves the machine. The cost is quality. The models are smaller than those of commercial services, and idiom and register are handled less well. Each sentence is translated on its own, with no memory of the conversation, so a pronoun that refers back to an earlier turn may be rendered wrongly. The packages are downloaded and installed on first startup, which needs a network connection once and adds several seconds to the first translation of a session.

8. API Design and Data Flow

The active FastAPI application exposes two implemented REST endpoints: POST /translate-audio and POST /reset.

8.1 Endpoints

POST /translate-audio processes one raw PCM audio request and returns the transcript, translation, and DoctorBrain replies. POST /reset replaces the global DoctorBrain instance, clearing its conversation state between consultations. This reset is global and does not provide per-user or per-session-isolated state.

8.2 Response contract

Field	Type	Meaning
user_transcript	string	German speech recognised by faster-whisper.
user_transcript_en	string	English translation of the learner transcript.
doctor_reply_de	string	Next German response from DoctorBrain.
doctor_reply_en	string	English response/subtitle

Field	Type	Meaning
		supplied by DoctorBrain.
translated_text	string	Deprecated duplicate of doctor_reply_en for compatibility.

8.3 Example response

```
{
 "user_transcript": "Ich habe Kopfschmerzen.",
 "user_transcript_en": "I have a headache.",
 "doctor_reply_de": "Nehmen Sie schon Schmerzmittel dagegen?",
 "doctor_reply_en": "Are you already taking painkillers for it?",
 "translated_text": "Are you already taking painkillers for it?"
}
```

8.4 Integration implications

The contract is deliberately simple for a prototype. A VR client can send audio and receive a predictable JSON object. For production use, session identifiers, authentication, request limits, streaming support, and stronger error semantics would be required.

9. Adaptive Learning Concept and Sensor Integration

Adaptation is the defining concept of the overall system. The intended system should vary the amount and type of support according to the learner's interaction and needs. However, the current AI server does not yet implement the complete sensor-driven adaptation loop. This section therefore distinguishes the target architecture from the current implementation.

Figure 3. Target adaptive-learning loop: interaction evidence, AI analysis, support decision, feedback and optional sensor/context input. The complete loop is future work.

9.1 Possible adaptation variables

Variable	Possible adaptation
Difficulty/support need	More or fewer hints and examples.
Pace	More time or slower interaction for learners who need it.
Repetition	Repeat a question or vocabulary item.
Feedback style	Shorter, clearer or more supportive feedback.
Audio/visual assistance	Add subtitles, highlighted words or spoken support.

9.2 Sensor integration

The wider project includes a separate sensor component, and Group B coordinated with the Sensor, XR and VR groups. Sensor/context information is a possible future input to adaptation. The AI-side requirement is that sensor uncertainty must not silently become a confident learner-state decision. The current prototype does not consume sensor state in /translate-audio, so the end-to-end sensor-to-AI adaptation claim is not made.

9.3 Future integration boundary

A future interface should pass a session-scoped learner-state object to the adaptation layer, including confidence/quality metadata. The AI should be able to fall back to interaction-only adaptation when sensor data is missing, interrupted or below a confidence threshold.

10. Doctor Scenario Walkthrough

The doctor scenario provides the clearest demonstration of the AI Support component. It is a controlled environment in which learners can practise communication before a real medical

interaction.

10.1 Scenario sequence

1. Enter the virtual doctor's-office scenario.
2. The learner listens to or reads the doctor's question.
3. The learner speaks a response in German.
4. The client sends the audio to the AI server.
5. faster-whisper produces a German transcript.
6. DoctorBrain identifies a symptom or conversational intent and selects the next response.
7. Argos Translate provides the English version of the learner transcript for subtitle/support purposes.
8. The structured JSON response is returned to the client.
9. The learner continues the conversation or receives a repeat request if speech was not understood.

10.2 Example conversation

Speaker	German	English support
Learner	Ich habe Kopfschmerzen.	I have a headache.
Doctor	Nehmen Sie schon Schmerzmittel dagegen?	Are you already taking painkillers for it?
Learner	Nein.	No.
Doctor	In Ordnung, manchmal ist das auch besser so.	All right, sometimes that's better anyway.

10.3 Learning value

The value of the scenario is not the medical diagnosis. The purpose is communication practice: describing symptoms, understanding questions, producing useful German phrases and gaining confidence in a realistic context. The system should therefore be presented as a language-learning simulation and not as a medical decision-making tool.

11. Testing and Evaluation

The current testing approach is prototype-oriented. It checks whether the main AI components behave as intended, while a full educational evaluation with A2 learners remains future work.

11.1 Technical test areas

Test area	Method	Current evidence/status
API request handling	Send valid PCM audio; verify JSON response.	Prototype/API documentation available.
Empty input	Send an empty request body.	400 response is explicitly implemented.
Speech recognition	Provide German speech and inspect transcript.	Functional path implemented; accuracy not statistically validated.
Symptom recognition	Use known symptom phrases.	Multiple symptom flows are implemented.
Conversation branching	Provide yes/no or symptom-specific answers.	Rule-based branches are implemented.
Translation	Translate German transcript to English.	Implemented with graceful fallback.
Unknown input	Use unrelated/vague phrases.	Generic responses are available.

11.2 Educational evaluation still needed

- Word/utterance recognition accuracy for non-native German speakers.
- Dialogue success rate across common A2 responses.
- Learner-perceived usefulness of translation and hints.

- Effect on confidence before and after practice.
- Accessibility and comfort of the future VR experience.
- Comparison of support levels for learners with different proficiency levels.

11.3 Evaluation principle

The project should avoid claiming learning effectiveness from technical functionality alone. A working API demonstrates engineering feasibility; it does not by itself demonstrate that learners improve their German. Educational claims should be supported by user testing with the target population.

11.4 End-to-End Test Results

The complete workflow was tested on 31 August 2026. A standalone test client, ai-server/tools/latency_test.py, records five seconds of audio from the system microphone at 16 kHz, converts it to 16-bit mono PCM, and posts it to /translate-audio. Timing code inside the endpoint measures speech recognition and translation separately. The client measures the round trip. The machine was an ordinary Windows laptop running the model on the CPU. Six requests were sent.

The pipeline worked. Every request returned a complete JSON response with the five documented fields. No request failed, and no connection was refused or dropped.

Run	Spoken input	Transcript	English	STT	Transl.	Server	Round trip
0	(silence)	Untertitel im Auftrag des ZDF, 2020	Subtitles on behalf of ZDF, 2020	6.56 s	5.05 s	11.61 s	13.67 s
1	Hallo, ich habe Kopfschmerzen.	correct	Hello, I have a headache.	1.70 s	0.05 s	1.75 s	3.79 s
2	Ich habe seit drei Tagen Fieber.	correct	I have had a fever for three days.	1.75 s	0.05 s	1.80 s	3.85 s
3	Achtunddreißig Grad.	38 Grad	38 degrees	1.59 s	0.03 s	1.62 s	3.67 s
4	Nein, ich rauche nicht.	correct	No, I don't smoke	1.63 s	0.04 s	1.67 s	3.73 s
5	Können Sie mir eine Krankschreibung geben?	correct	Can you give me a sick letter?	1.70 s	0.05 s	1.75 s	3.78 s

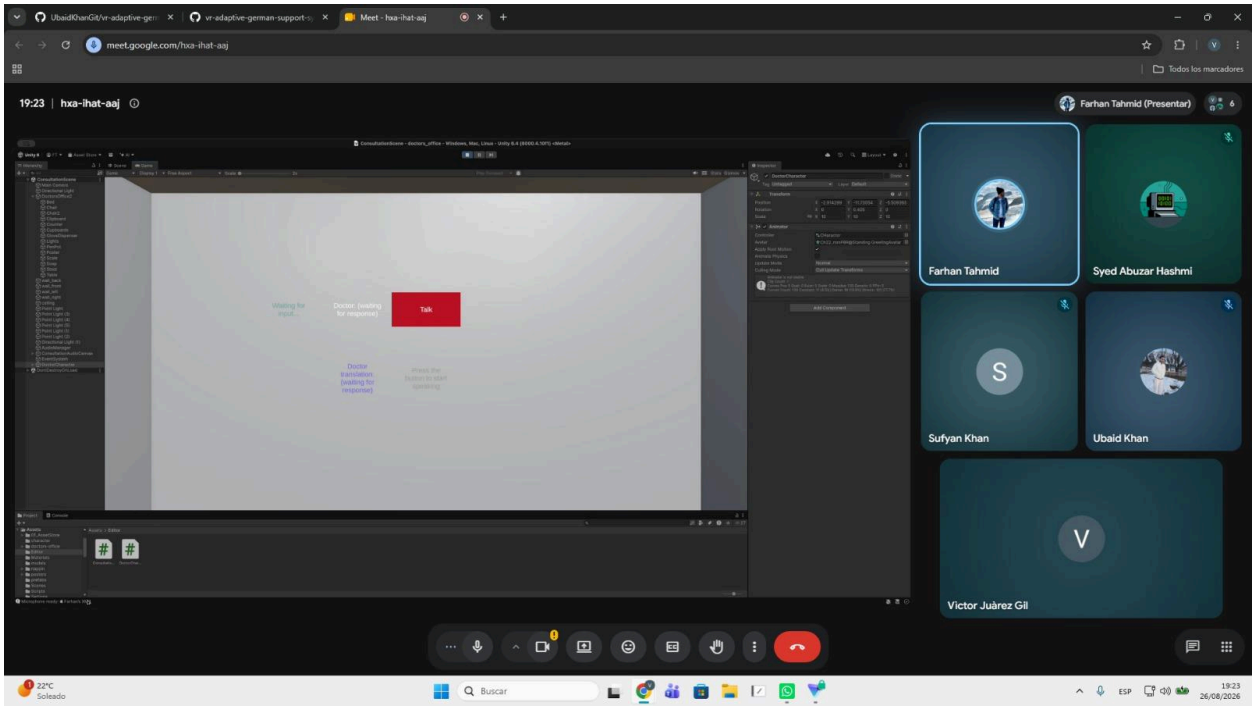
Latency. Once the server is warm, it takes about 1.7 seconds to process a five-second clip. Speech recognition accounts for nearly all of that. Translation takes between 30 and 50 milliseconds, which is small enough to ignore. The learner waits about 3.8 seconds in total, so roughly two seconds are spent outside the AI pipeline, in recording, transport and parsing.

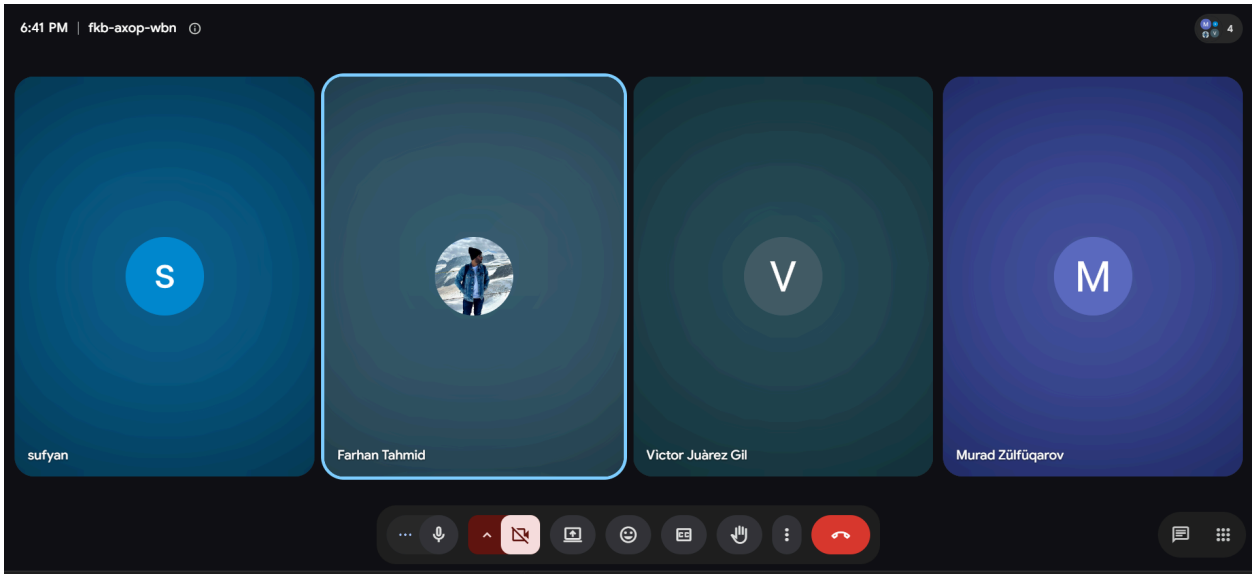
The first request is the exception. It took 11.61 seconds on the server and 13.67 seconds at the client. Whisper runs its first inference then, and Argos loads its model on first use, which alone took 5.05 seconds and never took more than 0.05 seconds again. The fix is plain enough: send one throwaway request when the server starts, and the learner never meets the delay.

Transcription. All five spoken utterances were transcribed correctly. The spoken number achtunddreißig came back as the digits 38, which the dialogue engine reads as a high-temperature answer, so the conversion helps rather than hinders.
Silence is another matter. Run 0 captured no speech, and the model returned Untertitel im Auftrag des ZDF, 2020, a subtitling credit that appears often in its training data. The transcript was not empty, so the server's repeat-request branch never fired, and the sentence was translated and answered as though the learner had said it. Whisper fails loudly and confidently instead of failing quietly. Checking the amplitude of the audio before transcribing it would catch this.
Translation. Argos handled all five sentences well enough for subtitle use. Krankschreibung came back as sick note, which is the correct term. The German language package asked for an extra segmentation component at startup, which Argos installed by itself and logged as a warning.
Dialogue. The most useful result was a fault, and it was not the one we expected. In run 1 the learner reported a headache, and DoctorBrain opened the headache flow. In run 2 the learner reported a fever. The engine was still inside the headache flow, so it read the sentence as an answer to its own question and asked where the pain was. Runs 3 and 4 were swallowed the same way, and the fever was never acknowledged at all. The state machine did what it was written to do. The conversation still went wrong, because a learner will not always answer the question that was asked. A learner who changes the subject is currently ignored, and at A2 level that is exactly what learners do. The engine needs to check for a new symptom before it treats an utterance as an answer.
Connection. No connection problems arose over localhost. The client and server ran on the same machine, so the network path was not tested. The Unity client currently addresses the server at a fixed local address, which will need changing before the pipeline can be reached from a headset on another device.

11.5 Interface/XR Integration Demonstration

A joint live coordination session was conducted with the Interface/XR development group via Google Meet. During this session, end-to-end communication was demonstrated by transmitting test audio and request inputs directly from the Unity-based XR interface (DoctorsOffice2 / ConsultationScene) to the AI server. The server successfully processed the incoming PCM audio, returned the German speech transcription, and delivered the corresponding English translation, which was rendered dynamically on the XR user interface canvas in real ti





12. Limitations and Future Work

12.1 Current limitations

- The /translate-audio processing path is single-shot; streaming/WebSocket communication is not implemented.

- DoctorBrain uses a single global state object rather than per-user sessions.
- Text-to-speech is demonstrated in the terminal prototype but is not wired into the FastAPI endpoint.
- Sensor data is not currently consumed by the AI endpoint as a complete adaptive signal.
- Speech-recognition accuracy for diverse A2 learners has not been established through a target-user study.
- The dialogue engine covers a limited set of scenarios and intents.
- The first request after startup takes about seven times longer than later ones, because the models load on first use.
- Silent or near-silent audio produces a confident but invented transcript rather than an empty one, so the empty-transcript path is not reached.
- While a symptom flow is active, a new symptom named by the learner is read as an answer to the current question instead of starting a new flow.

12.2 Future technical work

Area	Future work
Session management	Introduce a session ID and isolate DoctorBrain state per learner.
Streaming	Add WebSocket or another streaming mechanism for more natural conversation.
VR integration	Connect microphone input to the API and refine in-VR interactions.
TTS	Integrate reliable German text-to-speech into the server/client pipeline.
Sensor interface	Define a stable learner-state schema with confidence and interruption flags.
Adaptation engine	Implement support-level decisions based on validated interaction evidence.

Area	Future work
Scenario expansion	Add travel, Foreigners' Office and apartment scenarios.
Testing	Build an automated test suite and evaluate with real A2 learners.
Security	Add authentication, access controls, rate limits and secure deployment practices before production use.

12.3 Definition of completion

A future release should only be considered fully integrated when a learner can enter the VR scenario, speak naturally, receive a low-latency AI response, obtain appropriate language support, and complete a session without shared-state or integration errors. Sensor-informed adaptation should be treated as an additional feature that requires its own validation and responsible-use controls.

13. Team Contributions

13.1 Detailed individual contributions

Member	Detailed contribution
Md. Shaiful Azam (36068)	Contributed to the project from the initial planning and design stage and worked closely with the team throughout development. He helped design the AI pipeline and prepare the work packages, and contributed to speech-to-text, translation, server communication, and integration of AI output with the other project groups. He participated in meetings with the Sensor, XR and VR groups and contributed to the project presentation, coordination, testing and troubleshooting. His contribution combined technical development, system

Member	Detailed contribution
	integration, presentation, coordination and teamwork.
Muhammad Sufyan (34441)	Integrated and merged the Sensor Rule Engine logic into the main branch—enabling the AI server to evaluate features/ai_input_feature.csv by filtering quality_flag, calculating baseline_change, and dynamically setting gui_trigger and ai_action outputs alongside GET /sensor-engine/status and POST /sensor-data endpoints. Engineered the audio processing pipeline within ai-server/main.py, featuring an in-memory /translate-audio endpoint that processes 16 kHz, 16-bit mono PCM streams through Faster-Whisper German speech-to-text, DoctorBrain stateful dialogue management, and Argos Translate German-to-English translation.
Murad Zulfugarov (36296)	Contributed to core system architecture and dialogue-design decisions, developed the FastAPI backend–Unity integration interface, defined and validated the Unity–AI audio/API contract, added the POST /reset endpoint for resetting DoctorBrain's conversation state, performed backend/API testing and integration validation, identified and fixed backend error-handling and integration issues, and contributed to API and integration documentation.
Syed Abuzar Hashmi (34393)	Developed the API communication interface to seamlessly connect the Interface/XR application with the AI pipeline. Established data transmission specifications for audio/text inputs, structured real-time output formats for transcriptions and translations, and configured baseline connection parameters, settings, and error-handling routines. Additionally, co-authored comprehensive technical documentation covering the system architecture diagram, integrated tools and APIs, installation/execution instructions, testing procedures, interface integration guides, and known system limitations.
Ubaid Khan (34482)	I created and structured the project's GitHub repository and established the initial project structure, including the ai-server, vr-client, sensors, and docs folders. I organised and maintained the Group B documentation and contributed to documenting the AI architecture, project structure, and implementation status. I attended project meetings to stay informed about requirements, coordination, and project progress. I prepared and compiled the

Member	Detailed contribution
	final Group B report and supporting project documentation.
Víctor Juárez Gil (35433)	Designed and implemented the core AI pipeline of the DoctorBrain system. Built the complete speech-to-text stage, including microphone capture, audio buffering and silence/end-of-utterance detection, and integrated faster-whisper for German transcription. Integrated Argos Translate for German-English translation within the pipeline. Designed and implemented the DoctorBrain dialogue system itself, including the symptom conversation trees, the branch-matching logic, and the answering-mode behaviour that controls how the system responds to user input. Designed and implemented the pronunciation coaching feature together with the text-to-speech pronunciation modelling used to generate learner feedback (edge-tts). Partially responsible for the core design decisions concerning the dialogue model, the structure of the AI pipeline and the answering mode, which formed the basis for the subsequent backend and API integration work. Organised and scheduled some of the team meetings and handled coordination with the XR group. Participated in both project presentations, presenting the AI group's goals and progress alongside teammates and delivering his own sections in each; produced the complete slide deck for the second presentation. Authored the technical sections of the project documentation covering the AI pipeline, the dialogue system and the speech components.Instrumented the /translate-audio endpoint with per-stage timing and wrote a standalone test client (ai-server/tools/latency_test.py) that records German speech and posts it to the server. Carried out the end-to-end test of the complete pipeline, and documented the measured latency, cold-start behaviour, transcription and translation errors, and the dialogue-state limitation reported in Section 11.4.

13.2 Repository and collaboration

The group created a shared GitHub repository to separate the major technical areas. The repository includes ai-server, vr-client, sensors and docs. The README specifies feature branches, pull requests before merging and keeping the main branch stable.

13.3 Deliverables produced by Group B

- FastAPI AI backend prototype.
- Speech-to-text processing using faster-whisper.
- DoctorBrain rule-based dialogue engine.
- German-English translation support using Argos Translate.
- API documentation for /translate-audio and /reset.
- Architecture documentation and data-flow description.
- Standalone terminal prototype for testing dialogue and language assistance.
- Final technical report documenting implementation and limitations.

14. Conclusion and References

14.1 Conclusion

The Group B AI Support contribution establishes the language-processing foundation for the proposed VR-based German support system. The current prototype demonstrates a complete backend processing path from raw learner audio to German transcription, scenario-specific dialogue processing and English subtitle support. The architecture is intentionally modular so that it can later be connected to a Unity VR client and sensor-informed adaptation.